

Exercise 2

184.702 Machine Learning (VU 3,0) 2025W

Group 26

Vandák Gregor 12128763

Milic Lénárd 12509828

Nico Reiterer 51912112

Agenda

01

Datasets

02

Implementation

03

Comparison

04

Discussion

05

Conclusion / Lessons Learned

Datasets - Quick Overview

Chose datasets with different widths and sizes:

- **Large-scale Wave Energy Farm** (2020, [UCI](#)), *long & wide*
 - 63600 instances, 149 features (all numerical (98 integer, rest cont.))
- **Corn farming data** (2022, [Kaggle](#)), *small & not wide*
 - 422 instances, 22 features (15 categorical, 5 numerical, 2 geospatial)
- **Steel Industry Energy Consumption** (2021, [UCI](#)), *long & not wide*
 - 35040 instances, 11 features (1 date, 7 numerical, 3 categorical)

Dataset - Large-scale Wave Energy Farm

Size: 63 600 rows (no missing values) // 149 features

Features:

Coordinates of the 49 wave energy converters (X1, Y1, X2, Y2, ..., > 98 coordinates)

Individual converter power outputs

q-factor (some quality factor of converters)

Target variable: Total farm power output

Application possibilities:

Analysis of optimal WEC placement, prediction of output based on placement, ...

Dataset - Corn farming data

Size: 422 rows (no missing values) // 22 features

Features (subset):

Education levels (Primary, Secondary, Others)

Laborers (number of ppl, 2-8)

Fertilizer Amount (kg)

Acreage (acres)

Features with only one value:

Power & water source (*manual & rain*) // Crop Insurance (*No*) // Crop (*corn*) // County (*TAITA TAVETA*)

Target variable: Yield (kg per season per acre)

Application possibilities:

E.g., Agricultural Analysis - predicting corn yield using surveyed farming data + knowledge gain for improved yield

Dataset - Steel Industry Energy Consumption

Size: 35 040 rows (no missing values) // 11 features

Features:

- Usage_kWh (industry energy consumption)
- Reactive power (lagging/leading, kVarh)
- CO₂ emissions (tCO₂)
- Power factors (lagging/leading, %)
- Time variables: seconds from midnight, weekstatus (weekday/weekend), day of week

Target:

- Usage_kWh

Application possibilities:

- Energy usage/efficiency modeling
- Forecasting daily/monthly/annual consumption

Origin: DAEWOO Steel Co. Ltd in Gwangyang, South Korea. [\[UCI Link\]](#)

Paper: [Efficient energy consumption prediction model for a data analytic-enabled industry building in a smart city \(Sathishkumar et al., 2021\)](#)

Preprocessing steps

To make algorithms easy to compare on various datasets only few steps were done during preprocessing

- Selection of target variable
- Drop non informative features
- Remove rows with missing values
- One-hot encode categorical variables

Feature engineering was out of scope, since it would influence the results, giving a well-organized dataset higher accuracy.

Implementation – Individual approach

Each group member prepared their own implementation of the two algorithms

	Nico	Lenard	Gregor
Pros	Parallelization; native handling of categorical variables; SKlearn compatible	Class architecture; relatively fast	Fast on small to medium datasets; class architecture
Cons	Complicated; some parts not necessary; slower	No parallelization; mishandling of some edge cases	No parallelization; incompatible with SKlearn

Variance reduction methods

During development two approaches were in use.

Cumulative sum approach

- Use pre-calculated cumulative sum
 - Faster in execution time

Standard approach where we re-calculate variance after each split.

- Re-calculates variance for every potential split
 - Straight forward, but slow for larger datasets

Group member comparison on Corn dataset

Modell	Avg. RMSE	Avg. R2	Avg. Train Time (s)	Avg. predict time (s)
Sklearn RF	53.2357	0.8411	0.0159	0.0128
Nico RF	56.1605	0.8272	0.2981	0.1572
Gregor RF	53.3447	0.8393	0.0301	0.0107
Lenard RF	53.3415	0.8408	0.2405	0.0007

Implementation – Characteristics

Node class:

Stores split info (feature_idx, threshold, children, value)

DecisionTreeRegressor:

made sklearn-compatible (BaseEstimator, RegressorMixin) for usage of e.g. GridSearchCV()

Stopping criteria:

max_depth, min_samples_split, min_samples_leaf, zero variance

Feature subsampling:

Supports: None, int, float, 'sqrt', 'log2'

Implementation - Pseudocode

```
function grow_tree(X, y, depth):
```

```
    if stop_criteria: return Leaf(mean(y))
```

```
    feat_idx = select_features(n_features, max_features)
```

```
    best_split = find_best_split(X, y, feat_idx)
```

```
    if no_split: return Leaf(mean(y))
```

```
    left, right = partition(X, y, best_split)
```

```
    return Node(best_split, grow_tree(left), grow_tree(right))
```

```
function predict(X):
```

```
    return [traverse_tree(x, root) for x in X]
```

Implementation - Pseudocode

```
function traverse_tree(x, node): # x ... feature
    if leaf_node: return node.value
    if node.value <= threshold:
        return traverse_tree(x, node.left)
    else: return traverse_tree(x, node.right)
```

Comparison - Performance metrics

Mean Squared Error (MSE)

Penalizes large errors (quadratically)

Quite sensitive to outliers, highlights variance in predictions

$$\frac{1}{N} \sum_i^N (y_i - y_{pred_i})^2$$

Mean Absolute Error (MAE)

Linear penalty, (more) robust to outliers

$$\frac{1}{N} \sum_i^N |y_i - y_{pred_i}|$$

R² Score

Equals the proportion of variance explained by the model (1 = perfect fit, 0 = pred. the mean)

$$R^2 = 1 - \frac{\sum_i^N (y_i - y_{pred_i})^2}{\sum_i^N (y_i - y_{mean})^2}$$

Mean Absolute Percentage Error (MAPE)

Relative error measure (% deviation, in sklearn range 0-1)

Useful for scale-free, relative comparison

$$\frac{1}{N} \sum_i^N \left| \frac{y_i - y_{pred_i}}{y_i} \right|$$

Median Absolute Error (MedianAE)

Much more robust to outliers

$$\text{median} (|y_1 - y_{pred_1}|, \dots, |y_1 - y_{pred_1}|)$$

Comparison – Parameter experimenting

For finding optimal parameters for RF, GridSearchCV() was used with the following configurations and 5-fold CV:

- max_depth: [3, 7, None]
- min_samples_split: [2, 8]
- min_samples_leaf: [1, 7]
- n_estimators: [1, 5, 10]

For comparing the individual effects of each parameter, the following configurations were tested with 5-fold CV:

- max_depth: [3, 5, 10, 30, 50]
- n_estimators: [5, 10, 30, 50, 70]
- max_features: ["log2", "sqrt", 0.5, None]

Base parameters

n_estimators: 10
max_depth: None
min_samples_split: 2
max_features: None

NOTE: The number of experimented configurations was reduced for Wave Energy dataset to save time

Results on the test set - Corn dataset

Regressor	R2	MSE	MAPE	MAE	MedianAE	Training (s)	Prediction (s)
TreeCustom	0.660	5812.890	0.209	42.896	30.000	0.08338	0.00011
SklearnTree	0.632	6279.924	0.224	45.379	27.500	0.01664	0.00024
RFCustom	0.765	4017.796	0.220	40.234	27.700	11.18021	0.01336
SklearnRF	0.710	4950.876	0.187	39.679	27.035	0.77442	0.00042
SklearnGB	0.743	4394.675	0.187	37.677	25.751	0.13283	0.00043

CV results – Corn dataset

Regressor	R2	MSE	MAE	MedianAE	MAPE
TreeCustom	0.800 ± 0.034	3681.535 ± 120.990	42.041 ± 1.597	32.667 ± 6.200	0.205 ± 0.020
SklearnTree	0.795 ± 0.044	3775.553 ± 621.850	41.552 ± 3.872	28.000 ± 7.483	0.201 ± 0.008
RFCustom	0.812 ± 0.074	3628.710 ±1856.736	44.271 ± 9.553	34.060 ± 7.703	0.246 ± 0.065
SklearnRF	0.885 ± 0.022	2171.996 ± 566.374	35.521 ± 4.115	28.651 ± 3.099	0.191 ± 0.024
SklearnGB	0.852 ± 0.029	2792.6669± 659.926	40.088 ± 5.097	30.414 ± 5.849	0.221 ± 0.054

Parameter analysis – Corn dataset

GridSearchCV results:

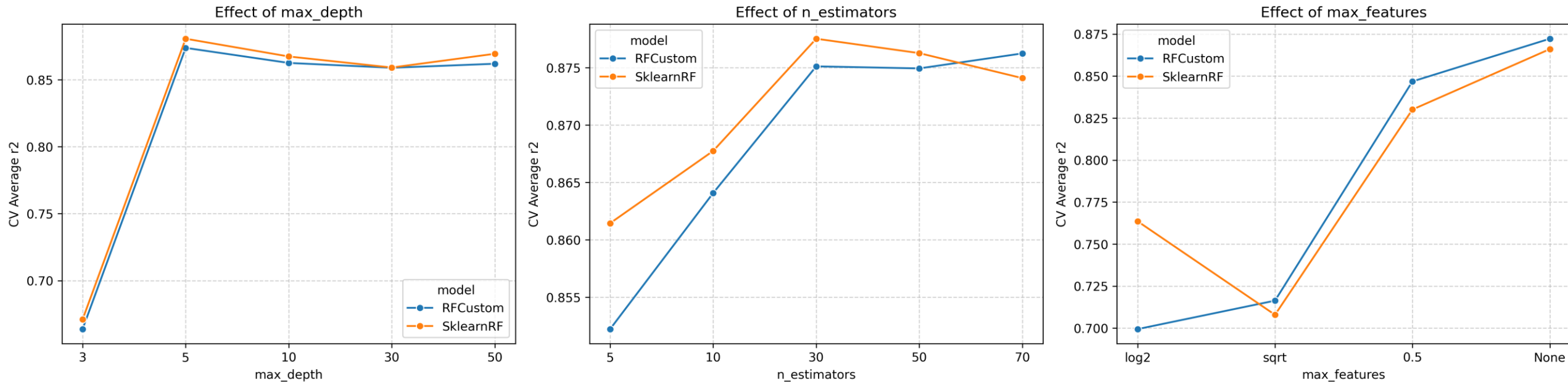
RFcustom:

max_depth: None, min_samples_leaf: 1, min_samples_split: 8, n_estimators: 10

SKlearnRF:

max_depth: 7, min_samples_leaf: 1, min_samples_split: 2, n_estimators: 5

Corn Yield - Model Sensitivity Analysis



Results on the test set - Steel dataset

Regressor	R2	MSE	MAE	MAPE	Training (s)	Prediction (s)
TreeCustom	0.995003	5.680386	1.20053	0.070263	3.714981	0.009548
SklearnTree	0.99797	2.307738	0.526377	0.016559	0.545017	0.00178
RFCustom	0.996957	3.459581	0.973588	0.043805	131.594	0.816595
SklearnRF	0.99869	1.489529	0.388853	0.013103	27.29266	0.012501
SklearnGB	0.99267	8.332455	1.761416	0.108572	9.534645	0.007183

CV results – Steel dataset

Regressor	R2	MSE	MAE	MAPE
TreeCustom	0.9953 ± 0.00031	5.2308 ± 0.4562	1.1539 ± 0.0452	$4.20\text{E}+11 \pm 8.39\text{E}+11$
SklearnTree	0.9979 ± 0.00021	2.3582 ± 0.2333	0.5589 ± 0.0172	$3.94\text{E}+11 \pm 7.87\text{E}+11$
RFCustom	0.9963 ± 0.00031	4.1157 ± 0.3596	1.0146 ± 0.0503	$4.48\text{E}+11 \pm 8.96\text{E}+11$
SklearnRF	0.9986 ± 0.00022	1.5163 ± 0.2386	0.4087 ± 0.0131	$3.97\text{E}+11 \pm 7.93\text{E}+11$
SklearnGB	0.9923 ± 0.00046	8.6154 ± 0.4048	1.7647 ± 0.0350	$3.98\text{E}+11 \pm 7.95\text{E}+11$

Parameter analysis – Steel dataset

GridSearchCV results:

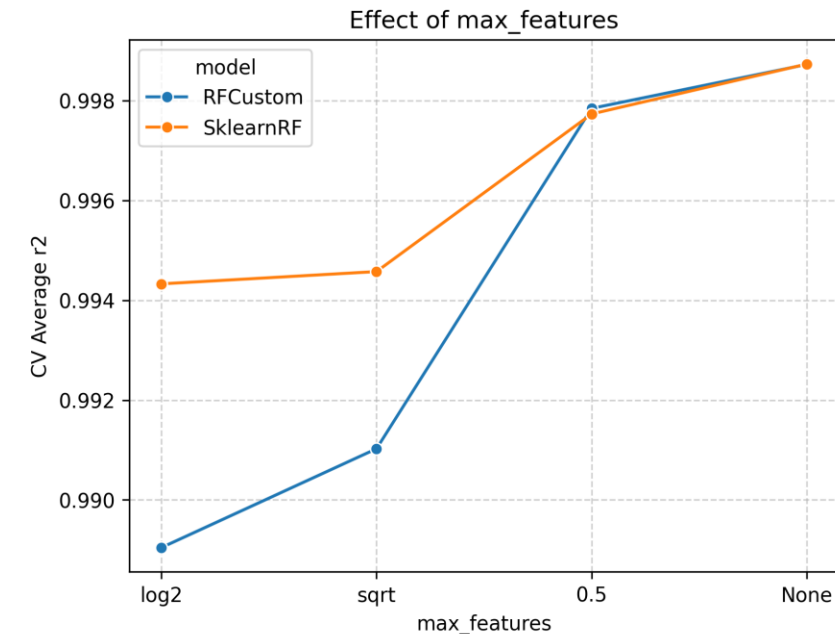
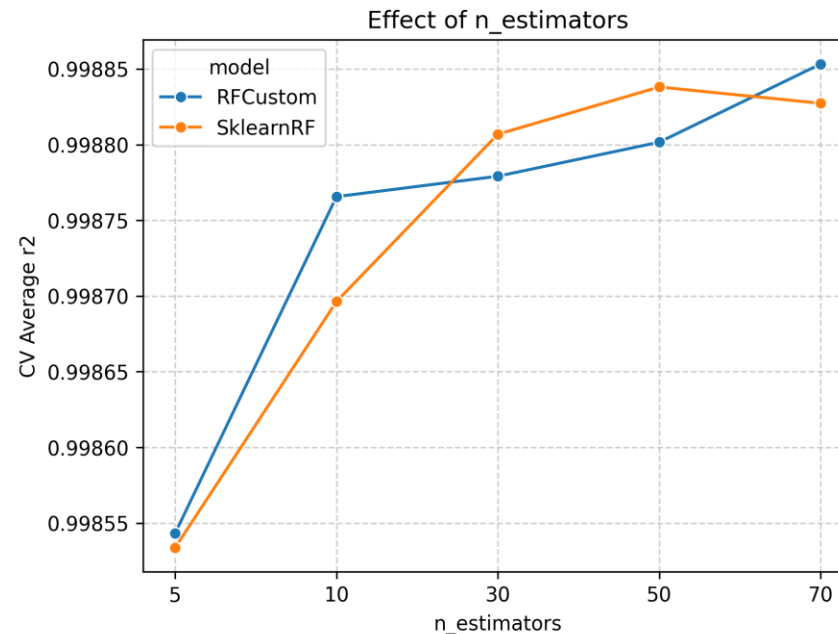
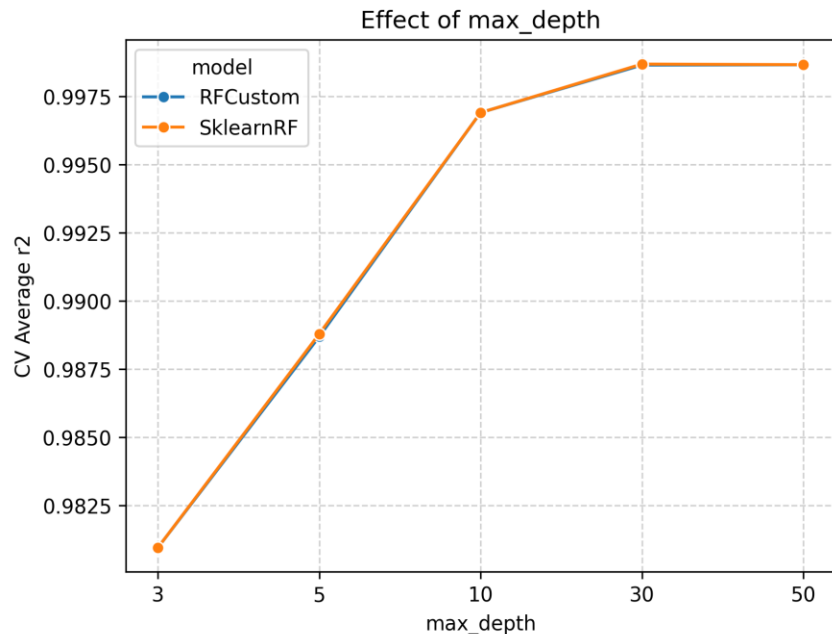
RFcustom:

max_depth: None, min_samples_leaf: 1, min_samples_split: 8, n_estimators: 10

SKlearnRF:

max_depth: None, min_samples_leaf: 1, min_samples_split: 2, n_estimators: 10

Steel Industry - Model Sensitivity Analysis



Results on the test set - Wave energy dataset

Regressor	r2	MSE	MAE	MAPE	Training (s)	Prediction (s)
TreeCustom	0.995132	71743596	5170.278	0.001332	46.5053	0.011702
SklearnTree	0.995692	63485803	2805.94	0.000724	7.074099	0.00251
RFCustom	0.995378	68113076	3093.285	0.000803	1107.389	1.005078
SklearnRF	0.997481	37121543	2677.775	0.000691	557.5853	0.016702
SklearnGB	0.992932	1.04E+08	7955.461	0.002037	281.8265	0.02943

CV results - Wave energy dataset

RRegressor	R2	MSE	MAE	MAPE
TreeCustom	0.9951 ± 0.00025	$74521167.0900 \pm 3554151.6794$	5124.8695 ± 99.1214	0.00132 ± 0.000025
SklearnTree	0.9955 ± 0.00028	$68554378.6599 \pm 4327361.2765$	2999.5114 ± 97.5832	0.00077 ± 0.000026
RFCustom	0.9951 ± 0.00043	$73442643.2477 \pm 6664437.5016$	3305.1251 ± 106.7311	0.00086 ± 0.000029
SklearnRF	0.9974 ± 0.00015	$39607930.1181 \pm 2348120.5835$	2808.2554 ± 82.4482	0.00073 ± 0.000022
SklearnGB	0.9929 ± 0.00020	$106774725.8155 \pm 2361686.8370$	8070.9679 ± 99.8111	0.00207 ± 0.000025

Parameter analysis - Wave energy dataset

GridSearchCV results:

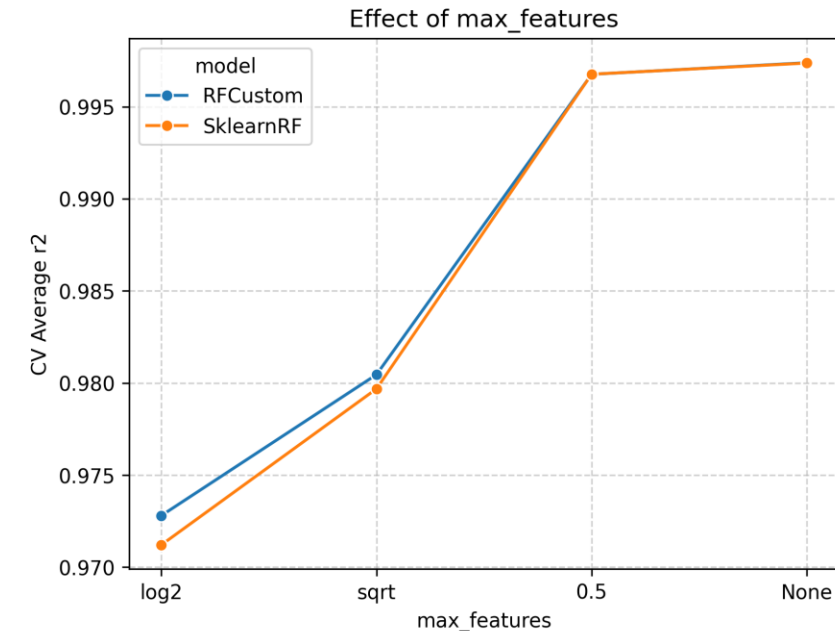
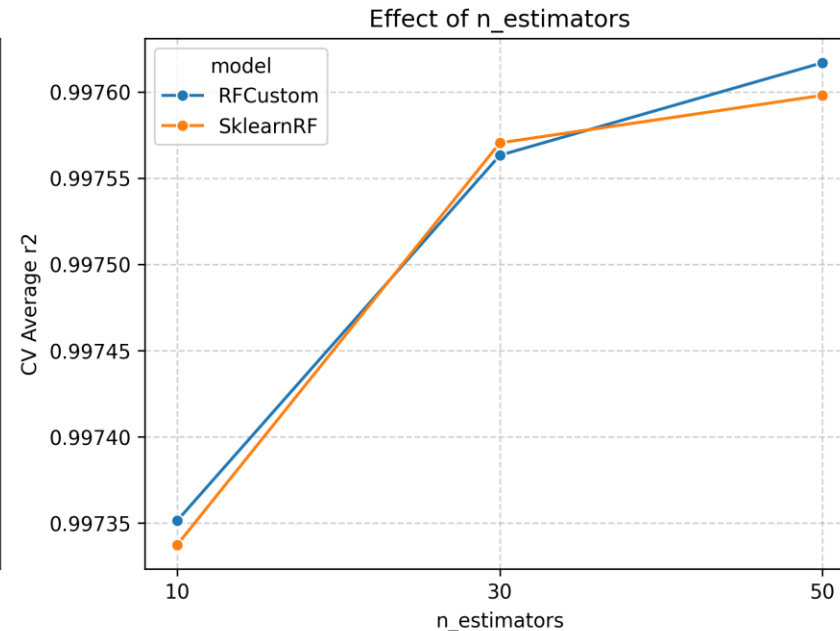
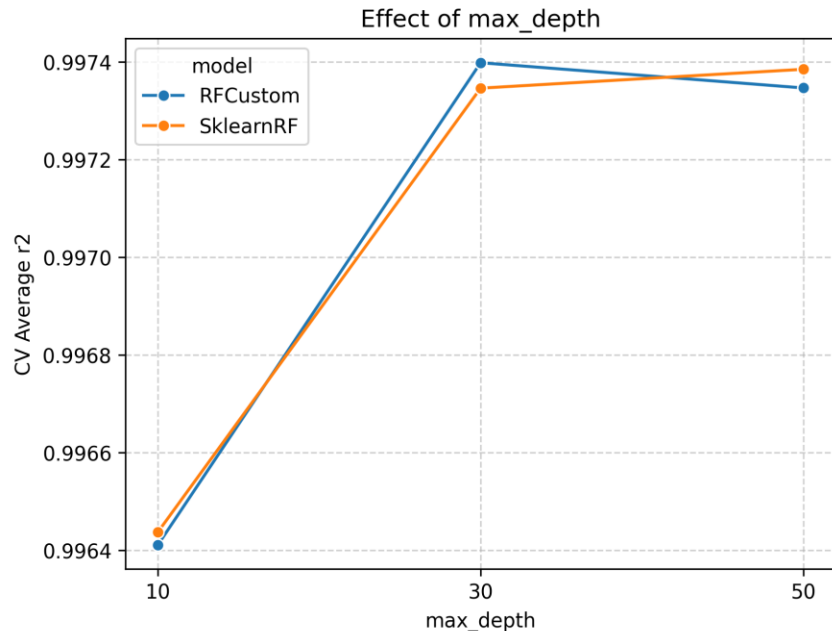
RFcustom:

max_depth: None, min_samples_leaf: 1, min_samples_split: 2, n_estimators: 10

SKlearnRF:

max_depth: None, min_samples_leaf: 1, min_samples_split: 8, n_estimators: 10

Wave Energy - Model Sensitivity Analysis



Discussion on differences in performance

Impurity definition

Ours: SSE (variance $\times$ n) (plain reason: SSE is easier with cumulative sums, avoided division until the end. Faster implementation, simpler math.)

Sklearn: normalized MSE -> Bias toward larger partitions, different split choices

Feature subsampling

Both: per-split random subset **but** RNG approach differs (default_rng in ours vs. RandomState in sklearn). That might lead to differences on the same data sets.

Pruning

Ours: None in this implementation to keep it minimal and interpretable

Sklearn: cost-complexity pruning (ccp_alpha)

-> Hence, our trees may overfit more, but forests should average it out to a certain degree

Discussion on differences in performance

Second Regressor we used – GradientBoostingRegressor*

- A method that builds trees sequentially and progresses each time.
- Each new tree reduces the error of the previous ones (gradient descent on the chosen loss function, default=squared error)
- Big downside: not parallelizable as each tree depends on the previous one.
- Prediction is also happening sequentially!

In anyway: Performance is generally higher with it if you have the resources/time to carefully finetune the model. With default values (as presented here) it sometimes already outperformed the trees/forests!

* Really good overview on it: <https://towardsdatascience.com/gradient-boosting-regressor-explained-a-visual-guide-with-code-examples-c098d1ae425c/>

Conclusion / Lessons Learned

Efficiency: Our implementation was slower than sklearn due to lots of optimization in the back with Cython and C/C++ code.

Efficient variance calculation is necessary for scalability and somewhat comparability to established frameworks.

Parallelization *necessary* for random forest (especially for larger datasets). Not hard to implement due to understandable Python API. Few things need to be considered (conditions must be reflected in separate methods, ...)

Thinking about “edge cases”: E.g., had to ensure at least one feature is selected (div by 0!)

Conclusion / Lessons Learned

Random feature selection must be robust to dataset size/shape (bootstrapping, forgot about that in the beginning)

Sklearn API compatibility is not hard to achieve and helpful for comparison (simple inheritance + naming conventions) and pays off in downstream usability with established frameworks (pipelines, CV, benchmarking)

Lots of different ways to create this algorithm.

Each of us implemented on their own in the beginning and we found multiple ways how to implement the algorithms (trees in dictionaries, node classes, categorical feature acceptance, ...)